SMART BEINGS · WOOHOO PLATFORM

Android
Integration Guide

Everything the in-room hub needs to talk to Woohoo V2 — the connection, the audio pipeline, the screen, offline behaviour, and what to delete from V1.

Kotlin | OkHttp | WebSocket | Working examples

FOR	APP	SERVICE
Android / device engineers	com.sb.woohoo	device-gateway 0.1.0

START HERE

The five-minute version

If you read nothing else, read this page. It is most of the integration.

What changes, and what doesn't

Component	Change	Why
Sensory wake word	KEEP	Works. Don't touch it. Re-tuning wake performance is a multi-month project with no product upside.
Audio capture / AEC	KEEP	Yours, and it must stay on the device.
Google STT call	KEEP FOR NOW	Works today. Moves server-side in phase 2 (§16). Not a blocker.
Dialogflow integration	DELETE	Replaced entirely. This is the bulk of the work — and it is deletion, not writing.
Where the transcript goes	CHANGE	One WebSocket to us instead. This is the whole integration.
AWS Polly on device	REPLACE	Speech comes back as text; synthesise it, or use the phrase pack (§9).
Cloud credentials in the APK	REMOVE	Highest-priority security item. Today a stolen device is a credential breach (§15).
socket.io channel	RETIRE	The same WebSocket carries push now. One connection, one auth model.

The entire protocol

```
// once, at boot
→ {"type":"hello","deviceId":"d_412a"}
← {"type":"ready","property":"Demo Hotel","room":"412","assistantName":"Voice Genie",
   "phrasePackVersion":"chirp3-en-in@en-IN"}

// per utterance, on the same open socket
→ {"type":"turn","transcript":"it's freezing in here","asrConfidence":0.94}
← {"type":"ack","filledPause":"Let me sort that—"}      ← play IMMEDIATELY
← {"type":"speak","text":"I've set it to 22.5 degrees.","turnId":"turn_…"}
← {"type":"screen","text":"22.5°C"}                     ← optional
```

That is the complete surface. Six message types out, five in.

THE THREE MISTAKES THAT WILL COST YOU

1. **Opening a connection per turn.** Three network round trips before a single byte moves — 200–400 ms on hotel Wi-Fi, on *every* interaction, against a total budget of about 1.1 s. Open it once at boot and keep it.
2. **Waiting for `speak` before making any sound.** `ack` arrives first, deliberately, and it is the difference between a device that feels instant and one the guest assumes is broken. Measured on our stack: **2 ms versus 1,846 ms** of silence.
3. **Hardcoding `asrConfidence`.** Below 0.60 the service asks the guest to repeat instead of guessing at a physical action. A constant means we guess when we shouldn't.

Contents

Part I — Foundations

1 Who owns what
2 Running the service locally
3 Message reference

Part II — The client

4 Complete WoohooClient
5 Connection lifecycle
6 Reconnection and jitter
7 Error handling

Part III — Audio

8 Wake word and the pre-roll buffer
9 Playback, ack and the phrase pack
10 Barge-in
11 Endpointing

Part IV — UI & state

12 The device state machine
13 Screen rendering
14 Confirmations and escalations

Part V — Production

15 Identity, certificates, credentials
16 Moving STT server-side
17 Offline mode
18 Telemetry

Part VI — Shipping

19 Testing against a local service
20 Migration plan
21 Review checklist

PART I

Foundations

1. Who owns what

The split is deliberate: **you own sound, we own meaning.** Nothing about understanding, deciding or authorising lives on the device, and nothing about microphones or speakers lives in the cloud.

Device (you)	Service (us)	Why there
Wake word detection	—	Privacy. No audio may leave the room before the wake phrase fires.
Echo cancellation, beamforming	—	Needs the raw mic array; must run before the signal leaves the mic path or barge-in is impossible.
Voice activity detection, endpointing	—	Signalling end-of-turn locally saves 200–400 ms versus a cloud silence timer.
Speech-to-text *(today)*	Speech-to-text *(phase 2)*	See §16.
—	Intent, reasoning, decisions	Changes weekly. Must not require an APK release.
—	Authorisation, spend limits, safety	Must be centrally auditable and instantly changeable.
—	Talking to hotel systems	Vendor differences must never reach the device.
—	Every word the assistant says	Wording is a compliance artefact in some paths (§14).
Playback, screen rendering	Screen *content*	Server decides what to show; device decides how.
Offline fallback	—	Must work with no network at all.

ONE RULE THAT EXPLAINS MANY OTHERS

The device never sends a room number, and never receives one it can act on. Identity is established once, at connection time, and the service derives tenant, property and room from it. If you ever find yourself wanting to put a room in a message, something has gone wrong — and there is nowhere in the protocol to put it.

2. Running the service locally

You do not need a hotel, an API key, a database or the internet.

```
git clone <repo> && cd woohoo
npm install
npm run gateway

# device-gateway listening on ws://localhost:8443
```

Three devices are seeded, and they differ on purpose so you can test capability behaviour:

deviceId	Room	Type	Use it for
`d_412a`	412	Deluxe King	A room with **no curtains, no scenes, no music** — test how absent capabilities behave
`d_511a`	511	Suite	Everything available
`d_512a`	512	Suite	Suite whose curtains are disabled by config — tests per-room overrides

Quickest possible smoke test, no app required:

```
curl -X POST http://localhost:8443/v1/turn \
  -H 'content-type: application/json' \
  -d '{"deviceId":"d_412a","transcript":"turn on the lights"}'

{"turnId":"turn_…","speech":"Done.","lane":"fast","totalMs":39,
 "calls":[{"tool":"lighting.set","tier":"T1","verdict":"ALLOW","ok":true}]}
```

From your emulator, `localhost` is `10.0.2.2`. For a physical device on the same Wi-Fi, use the machine's LAN IP.

3. Message reference

3.1 You send

type	Fields	When	Notes
`hello`	`deviceId` req · `firmware` · `certificateSerial`	First frame, always	Anything sent before this is rejected with `NOT_AUTHENTICATED`
`turn`	`transcript` req · `asrConfidence`	Per utterance	The main message
`barge_in`	—	Guest speaks over playback	Stop playback locally too — don't wait for us
`health`	`rssi` · `wakeFalseAccepts`	Every few minutes	A rising false-accept count usually means the device sits next to a TV
`ping`	—	Optional	Answered with `pong`. Separate from protocol-level pings

3.2 You receive

type	Fields	What to do
`ready`	`property` `room` `assistantName` `phrasePackVersion`	Store them. Use `assistantName` for branding, `phrasePackVersion` to decide whether to refresh cached audio
`ack`	`filledPause`	**Play immediately.** Do not wait for `speak`
`speak`	`text` `turnId`	Synthesise and play. Always short — capped around 45 words
`screen`	`text`	Render. May contain `\n`. Present only when the display carries something the audio does not
`error`	`code` `detail`	See §7
`pong`	—	Liveness reply

> **THERE IS NO REQUEST ID, AND YOU DON'T NEED ONE**
>
> The connection is per-device and turns are serialised — one `turn` in, one `ack` plus one `speak` out. The `turnId` on the response is what you log against. Do not send a second `turn` before the first has produced `speak`; queue it or drop it.

PART II

The client

4. Complete WoohooClient

Production-shaped, using OkHttp, which is already in your `build.gradle`. No new dependency.

```kotlin
package com.sb.woohoo.net

import android.os.Handler
import android.os.Looper
import okhttp3.*
import org.json.JSONObject
import java.util.concurrent.TimeUnit
import java.util.concurrent.atomic.AtomicBoolean

/**
 * Single long-lived connection to the Woohoo service.
 *
 * Lifecycle: connect() once when the app starts, disconnect() when it stops.
 * NOT one connection per utterance — see §5.
 */
class WoohooClient(
    private val deviceId: String,
    private val endpoint: String,
    private val firmware: String,
    private val listener: Listener,
) {
    interface Listener {
        /** Identity accepted. Safe to accept utterances from here. */
        fun onReady(property: String, room: String, assistantName: String, phrasePackVersion: String)
        /** Play from the local phrase pack IMMEDIATELY. Do not wait for onSpeak. */
        fun onAck(filledPause: String)
        /** The real answer. Synthesise and play. */
        fun onSpeak(text: String, turnId: String)
        fun onScreen(text: String)
        fun onError(code: String, detail: String, fatal: Boolean)
        fun onConnectionStateChanged(state: State)
    }

    enum class State { DISCONNECTED, CONNECTING, READY }

    private val http = OkHttpClient.Builder()
        // Hotel Wi-Fi drops connections without a close frame. Keepalives detect it.
        .pingInterval(20, TimeUnit.SECONDS)
        .connectTimeout(10, TimeUnit.SECONDS)
        .readTimeout(0, TimeUnit.SECONDS)   // 0 = never; the socket is long-lived
        .retryOnConnectionFailure(true)
        .build()

    private var ws: WebSocket? = null
    private var retries = 0
    private val shuttingDown = AtomicBoolean(false)
    private val main = Handler(Looper.getMainLooper())

    var state: State = State.DISCONNECTED
        private set(value) {
            field = value
            main.post { listener.onConnectionStateChanged(value) }
        }

    val isReady: Boolean get() = state == State.READY

    // ─────────────────────────────────────────────────────── lifecycle

    fun connect() {
        if (shuttingDown.get() || state != State.DISCONNECTED) return
        state = State.CONNECTING

        val request = Request.Builder().url(endpoint).build()
        ws = http.newWebSocket(request, object : WebSocketListener() {

            override fun onOpen(webSocket: WebSocket, response: Response) {
                // hello MUST be first. Everything else is rejected until identity exists.
                webSocket.send(
                    JSONObject()
                        .put("type", "hello")
```

```
                    .put("deviceId", deviceId)
                    .put("firmware", firmware)
                    .toString()
            )
        }

        override fun onMessage(webSocket: WebSocket, text: String) {
            handle(text)
        }

        override fun onFailure(webSocket: WebSocket, t: Throwable, response: Response?) {
            state = State.DISCONNECTED
            scheduleReconnect()
        }

        override fun onClosed(webSocket: WebSocket, code: Int, reason: String) {
            state = State.DISCONNECTED
            if (!shuttingDown.get()) scheduleReconnect()
        }
    })
}

fun disconnect() {
    shuttingDown.set(true)
    ws?.close(1000, "client shutdown")
    ws = null
    state = State.DISCONNECTED
}

// ──────────────────────────────────────────────────── sending

/**
 * Send one transcribed utterance.
 *
 * @param asrConfidence the REAL score from the recogniser. Below 0.60 the service asks
 *        the guest to repeat rather than guessing at a physical action; a hardcoded
 *        constant makes it guess when it shouldn't.
 * @return false if not connected — show the offline affordance (§17).
 */
fun sendTurn(transcript: String, asrConfidence: Double): Boolean {
    if (!isReady) return false
    if (transcript.isBlank()) return false
    return ws?.send(
        JSONObject()
            .put("type", "turn")
            .put("transcript", transcript.take(2000))
            .put("asrConfidence", asrConfidence.coerceIn(0.0, 1.0))
            .toString()
    ) ?: false
}

fun sendBargeIn() {
    ws?.send(JSONObject().put("type", "barge_in").toString())
}

fun sendHealth(rssi: Int, wakeFalseAccepts: Int) {
    ws?.send(
        JSONObject()
            .put("type", "health")
            .put("rssi", rssi)
            .put("wakeFalseAccepts", wakeFalseAccepts)
            .toString()
    )
}

// ──────────────────────────────────────────────────── receiving

private fun handle(text: String) {
    val msg = try { JSONObject(text) } catch (e: Exception) { return }
    main.post {
        when (msg.optString("type")) {
```

```
                "ready" -> {
                    retries = 0
                    state = State.READY
                    listener.onReady(
                        msg.optString("property"),
                        msg.optString("room"),
                        msg.optString("assistantName", "Voice Genie"),
                        msg.optString("phrasePackVersion"),
                    )
                }
                "ack"    -> listener.onAck(msg.optString("filledPause"))
                "speak"  -> listener.onSpeak(msg.optString("text"), msg.optString("turnId"))
                "screen" -> listener.onScreen(msg.optString("text"))
                "error"  -> {
                    val code = msg.optString("code")
                    // UNKNOWN_DEVICE means the socket is closed and retrying is pointless —
                    // this is a provisioning problem, not a network one.
                    listener.onError(code, msg.optString("detail"), fatal = code == "UNKNOWN_DEVICE")
                    if (code == "UNKNOWN_DEVICE") shuttingDown.set(true)
                }
            }
        }
    }

    // ──────────────────────────────────────────────────────── reconnection

    /**
     * Exponential backoff with FULL jitter, capped at 60s.
     *
     * The jitter is not optional. Without it, a property-wide Wi-Fi blip makes 300 devices
     * reconnect in lockstep and you have caused your own outage. Random across the whole
     * window, not "base + random" — that still clusters.
     */
    private fun scheduleReconnect() {
        if (shuttingDown.get()) return
        val capMs = minOf(1000L shl minOf(retries, 6), 60_000L)
        retries++
        val delay = (Math.random() * capMs).toLong().coerceAtLeast(250L)
        main.postDelayed({ connect() }, delay)
    }
}
```

5. Connection lifecycle

```
app start ──► connect() ──► onOpen ──► send hello ──► ready ──► State.READY
                  ▲                                              │
                  │                                              │ utterances flow
                  │                                              │ here, many of them,
                  │                                              │ on the SAME socket
                  │                                              ▼
                  └──── scheduleReconnect() ◄──── onFailure / onClosed
                        (jittered backoff)

app stop ──► disconnect()      shuttingDown = true, no reconnect
```

Bind it to the process, not to an activity or a turn. On a dedicated in-room device the natural home is a foreground service that lives as long as the app.

```
class VoiceService : Service(), WoohooClient.Listener {
    private lateinit var client: WoohooClient

    override fun onCreate() {
        super.onCreate()
        client = WoohooClient(
            deviceId = DeviceIdentity.current(this),
            endpoint = BuildConfig.WOOHOO_WS_URL,
            firmware = BuildConfig.VERSION_NAME,
            listener = this,
        )
        client.connect()   // once. not per turn.
    }

    override fun onDestroy() {
        client.disconnect()
        super.onDestroy()
    }
}
```

> **WHY ONE CONNECTION MATTERS, WITH THE ARITHMETIC**
>
> Before a single byte moves, opening a connection costs **three network round trips**: TCP handshake, TLS handshake, WebSocket upgrade. On hotel Wi-Fi a round trip is typically 40–100 ms, so that is **120–300 ms of pure overhead** — paid on every utterance, forever, buying nothing.
>
> Against a p50 budget of about 1,100 ms wake-to-response, that is a quarter of your budget spent dialling. Open it once.

6. Reconnection and jitter

Situation	Behaviour
Socket drops	Reconnect with jittered backoff, 250 ms → 60 s cap. Re-send `hello`.
Reconnect within 30 min	Conversation context survives server-side, keyed on the room. A brief drop mid-conversation loses nothing.
Reconnect after a pending confirmation	The frozen action expires after **90 s** regardless. If the guest says "yes" later, they are told it expired and nothing is charged.
`UNKNOWN_DEVICE`	**Stop.** This is provisioning, not networking. Retrying forever hides the real fault. Show a service state on screen.
Offline for a long period	Enter offline mode (§17). Keep trying in the background at the 60 s cap.

7. Error handling

code	Meaning	Do
`NOT_AUTHENTICATED`	A frame arrived before `hello`	Client bug. Send `hello`, retry once.
`UNKNOWN_DEVICE`	Unknown, revoked, or room binding broken	**Fatal.** Socket closes. Stop retrying; show a service message.
`RATE_LIMITED`	>30 turns from this device in an hour	Say something apologetic; back off a few minutes.
`BAD_FRAME`	Not valid JSON	Client bug. Log it.
`TURN_FAILED`	The turn threw server-side	Say something honest and offer the front desk. Do not auto-retry — it may have half-executed.

NEVER INVENT AN OUTCOME

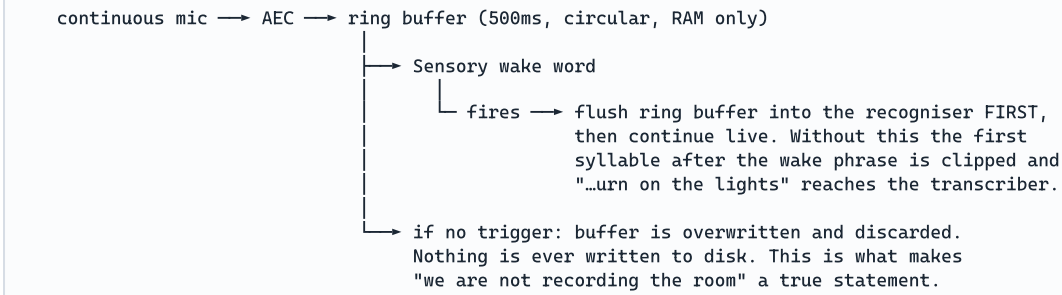
On any error, do **not** say "Done". The single most damaging failure mode in this product is the confident false confirmation: the guest hears "I've turned off the lights", the lights stay on, and the device is dead to them. If you don't have a `speak`, say you couldn't do it.

PART III

Audio

8. Wake word and the pre-roll buffer

Keep Sensory. What matters is what happens in the 500 ms *around* the trigger.




```
class PreRollBuffer(private val capacityMs: Int = 500, sampleRate: Int = 16_000) {
    private val capacity = capacityMs * sampleRate / 1000
    private val buf = ShortArray(capacity)
    private var writeIdx = 0
    private var filled = 0

    fun write(samples: ShortArray, len: Int) {
        for (i in 0 until len) {
            buf[writeIdx] = samples[i]
            writeIdx = (writeIdx + 1) % capacity
            if (filled < capacity) filled++
        }
    }

    /** Oldest-to-newest snapshot, for flushing on wake. */
    fun drain(): ShortArray {
        val out = ShortArray(filled)
        val start = if (filled < capacity) 0 else writeIdx
        for (i in 0 until filled) out[i] = buf[(start + i) % capacity]
        clear()
        return out
    }

    /** No trigger — the audio is gone. Called continuously. */
    fun clear() { filled = 0; writeIdx = 0 }
}
```

PRIVACY PROPERTIES YOU MUST PRESERVE

- The ring buffer lives in **RAM only**. Never write it to storage, not even for debugging, not even behind a flag.
- Nothing leaves the device until the wake word fires.
- The hardware mute must cut the microphone **in the driver**, not set a boolean. A software mute cannot be verified by a guest, and a guest who doesn't trust the mute stops using the product.
- The privacy LED should not be drivable by app code alone.

9. Playback, ack and the phrase pack

This section is the difference between a device that feels instant and one guests assume is broken. It is worth more than any latency optimisation in the backend.

9.1 Measured, on our own stack

```
// realistic model latency
guest stops speaking          at    0ms
ack   "Let me sort that—"     at    2ms   ← the device speaks HERE
speak "I've set it to 22.5"   at 1846ms

silence the guest experiences :    2ms
silence WITHOUT the ack       : 1846ms   (877x longer)
```

Identical work, identical total time. Completely different experience.

9.2 How to implement it

`filledPause` is a **short phrase you already have audio for**. It is not something to synthesise on receipt — synthesising it would defeat the entire purpose.

```
class PhrasePack(private val ctx: Context) {
    private val cache = mutableMapOf<String, Uri>()

    /** Pre-synthesised WAV/OGG shipped or fetched per (voice, language). */
    fun lookup(phrase: String): Uri? = cache[normalise(phrase)]

    fun play(phrase: String): Boolean {
        val uri = lookup(phrase) ?: return false
        AudioPlayer.playImmediate(uri)   // <20ms
        return true
    }

    private fun normalise(s: String) = s.trim().lowercase().trimEnd('-', '–', '…', '.')
}
```

```
override fun onAck(filledPause: String) {
    // 1. cached audio — the fast path, and the one that matters
    if (phrasePack.play(filledPause)) return

    // 2. not in the pack (new phrase, or pack not refreshed yet):
    //    a soft earcon still beats silence. Do NOT synthesise — TTS round-trip
    //    is exactly the delay this whole mechanism exists to hide.
    AudioPlayer.playEarcon(R.raw.thinking_chime)
}

override fun onSpeak(text: String, turnId: String) {
    tts.speak(text, TextToSpeech.QUEUE_ADD, null, turnId)
}
```

> **BUILDING THE PACK**
>
> The service rotates through six acknowledgements per language. Pre-synthesise all of them, per property voice, and ship or fetch on provisioning. `phrasePackVersion` in `ready` tells you when to refresh — compare against what you cached and re-fetch on change.
>
> Worth caching beyond acknowledgements: "*Done.*", "*Certainly.*", "*I'm connecting you to the front desk now.*", and the numbers 16–30 for temperatures. Those cover a large share of everything the assistant ever says.

9.3 Two rules

- **Only fill when you will be slow.** If `speak` arrives within about 250 ms of `ack`, the pause makes a fast interaction feel slower. Consider suppressing playback if `speak` has already arrived when the audio would start.
- **Never let a filled pause claim an outcome.** "*One moment*" is honest. "*Done!*" before the room system has confirmed is a lie the guest catches when the lights don't change.

10. Barge-in

Interrupting a talkative assistant is the single most-requested behaviour in voice UX.

```
class BargeInController(
    private val player: AudioPlayer,
    private val client: WoohooClient,
) {
    /** Called by VAD while the assistant is speaking. AEC must already be running,
     *  or the assistant hears itself and barges in on its own voice. */
    fun onVoiceDetectedDuringPlayback() {
        player.duck()                    // 1. duck first — avoids false trips
        mainHandler.postDelayed({
            if (vad.stillSpeaking()) {
                player.stop()            // 2. genuine interruption
                client.sendBargeIn()     // 3. tell the server
                startListening()         // 4. capture the new utterance
            } else {
                player.unduck()          // false alarm
            }
        }, 120)
    }
}
```

Stop playback **locally and immediately**. Do not wait for a server response — the guest is already talking over you.

11. Endpointing

Detecting end-of-speech on the device rather than letting the cloud wait out a silence timer is worth 200–400 ms per turn, and it is free.

Setting	Start at	Notes
Trailing silence threshold	380 ms	The main knob. Lower feels snappier but truncates slow speakers.
Max utterance length	15 s	Hard stop. Prevents a stuck mic streaming forever.
Min utterance length	300 ms	Below this it is almost always a false wake.
Accessibility mode	900 ms	Per-room setting staff can enable at check-in for guests who speak slowly.

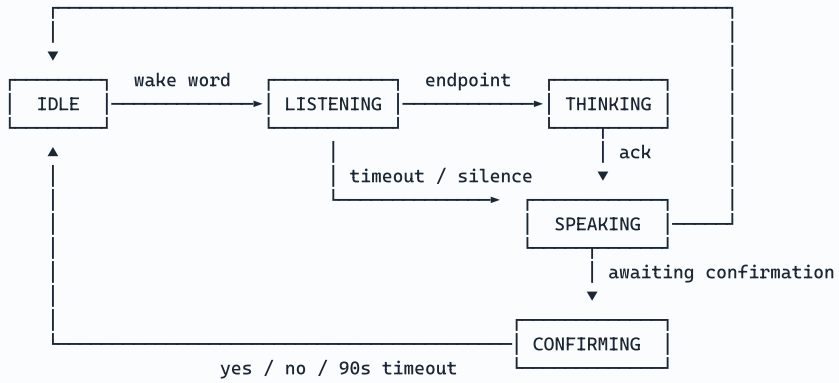
TUNE THIS ON REAL HARDWARE, IN A REAL ROOM

Endpointing is the UX/accuracy trade-off with the most user-visible consequence, and it cannot be tuned at a desk. A room next to a lift shaft behaves differently from a quiet corner room. Make the threshold remotely configurable per property so it can be adjusted without an APK.

PART IV

UI & state

12. The device state machine




```
Cross-cutting, can be entered from anywhere:
  MUTED     hardware switch — mic dead, LED on
  OFFLINE   no connection — local grammar only (§17)
  SERVICE   fatal error, e.g. UNKNOWN_DEVICE
```

State	Entered by	Screen shows	Mic
IDLE	Default	Time, room, ambient info	Wake word only
LISTENING	Wake word fires	Clear "I'm listening" affordance	Capturing
THINKING	Endpoint detected	Activity indicator	Off
SPEAKING	`ack` or `speak`	`screen` content if present	On for barge-in
CONFIRMING	A chargeable action is pending	**The total, prominently**	Listening for yes/no
MUTED	Hardware switch	Unambiguous mic-off indicator	**Dead in hardware**
OFFLINE	No connection	Offline badge + what still works	Local grammar

13. Screen rendering

Content is server-driven; presentation is yours. The device renders, it never decides.

```
override fun onScreen(text: String) {
    // May contain \n. Never markdown. Render as plain lines.
    screenState.showMessage(text.split("\n"))
}
```

> **ACCESSIBILITY IS NOT OPTIONAL HERE**
>
> Voice must never be the only path. Guests who are deaf, hard of hearing, non-verbal, or who have atypical speech must be able to reach every function through the screen. This is an obligation under ADA and EN 301 549, and it is simply correct.
>
> Practically: always render `speak` text on screen as well as playing it; meet contrast requirements; support text scaling; and never make a control reachable by voice alone. In an emergency the screen works when speech does not — which is exactly when it matters.

14. Confirmations and escalations

14.1 Chargeable actions take two turns

```
→ turn  "I'd like a club sandwich and a beer"
← ack   "One second—"
← speak "I have 2 items ready to order. That's 1240 INR, charged to your room.
         Shall I place it?"
← screen "I have 2 items ready to order. That's 1240 INR, charged to your room."

// nothing has been ordered. nothing has been charged.

→ turn  "yes"
← speak "Ordered — about 30 minutes. 1240 INR has gone to your room bill."
```

Your responsibilities in `CONFIRMING`:

- **Show the total prominently.** The guest is agreeing to a charge; the screen is the record they can actually read.
- **Never auto-confirm.** Not on timeout, not on ambiguity, not ever.
- Offer a visible cancel affordance.
- After ~90 s, drop the state. The server expires the frozen action independently, so a late "yes" is safe — the guest is told it expired and nothing is charged.

14.2 Escalations

```
→ turn  "I'm having chest pain"
← speak "I'm alerting the hotel team right now. If this is an emergency please call 112 —
         I'm not able to call them for you. I'll stay here with you."
← screen "Emergency: 112
          Hotel team alerted"
```

> **DO NOT MODIFY, SHORTEN OR PARAPHRASE ESCALATION WORDING**
>
> That text is a compliance artefact, not copy. It says plainly what the device *cannot* do, because a guest who believes help is coming when it isn't is the worst outcome this system can produce.
>
> On the device: render the screen text large and persistently — do not auto-dismiss after a few seconds. Keep the microphone open rather than returning to idle. Never add your own reassurance on top.

PART V

Production concerns

15. Identity, certificates, credentials

Aspect	Today	Production target
Identity	`deviceId` string in `hello`	Per-device X.509 certificate; the ID comes from the certificate subject and the `hello` field is ignored
Key storage	—	Android Keystore / StrongBox, non-exportable
Transport	`ws://`	`wss://`, TLS 1.3, mTLS, pinning *with a rotation path*
Cloud credentials in APK	**Present — remove**	None. Ever.
Provisioning	—	QR scan (ZXing, already present) → generate keypair in hardware → CSR → signed cert scoped to one room

> **THE HIGHEST-PRIORITY CHANGE IN THIS ENTIRE MIGRATION**
>
> The current APK ships `google-services.json` with a service-account path, and AWS Polly keys. A device stolen from a guest room — which happens — is therefore a **credential compromise**, and the credential is not limited to that room.
>
> Moving STT and TTS server-side removes them as a side effect. That is the real security argument for doing it, and it is the finding that would fail the first serious hotel-chain security review.

```
// Certificate pinning MUST ship with a rotation plan.
// Pinning without one is how fleets get bricked at 2am.
val pinner = CertificatePinner.Builder()
    .add("gateway.woohoo.example", "sha256/CURRENT_PIN=")
    .add("gateway.woohoo.example", "sha256/BACKUP_PIN=")  // next cert, pre-published
    .build()
```

16. Moving STT server-side

Phase 2, not a blocker. Today you send text and that is fine.

Aspect	Device STT (today)	Cloud STT (target)
Accented speech	Whatever the on-device path gives	Materially better — hotel guests are the most accent-diverse audience of any voice product
Hotel vocabulary	Cannot be biased per property	Phrase hints per property: menu items, staff titles, local landmarks
Language switching	APK change	Config change
Improving accuracy	Fleet-wide OTA	Deployed in minutes
Credentials on device	**Yes — the problem**	None

When it lands, the change on your side is: instead of `{"type":"turn","transcript":…}` you stream Opus binary frames between wake and endpoint, and the `ack` / `speak` flow is unchanged. Target format is **Opus, 16 kHz mono, 20 ms**

frames, ~16 kbps, DTX on. Nothing else about the client changes.

17. Offline mode

Hotel WANs fail, often at night during maintenance. The device must not become a brick with a logo.

```
  connected     ──► full capability
       │
       │ socket down
       ▼
  OFFLINE       ──► announce ONCE, then:
                    · local grammar (~25 phrases) over the LAN to the property appliance
                    · lights · temperature · do-not-disturb · curtains · TV
                    · everything else: honest refusal + point at the room phone
                    · service requests queued locally, guest TOLD they are queued
```

```
override fun onConnectionStateChanged(state: WoohooClient.State) {
    when (state) {
        DISCONNECTED -> if (!offlineAnnounced) {
            // Say what you CAN do, not what is broken. This keeps the guest using
            // the product; "sorry, I'm having trouble" teaches them to stop trying.
            tts.speak("I'm offline right now — I can still do lights, temperature " +
                      "and do not disturb.")
            screenState.showOfflineBadge()
            offlineAnnounced = true
        }
        READY -> { offlineAnnounced = false; screenState.clearOfflineBadge() }
        else -> {}
    }
}
```

> **ONE DEPENDENCY WORTH FLAGGING EARLY**
>
> The offline path needs the property appliance to expose a LAN listener for devices. That listener is **not built yet** — the appliance is currently outbound-only. Until it exists, offline mode can show the badge and refuse honestly, but cannot actuate. Plan the UI for it now; wire the actuation when the listener lands.

18. Telemetry

```
client.sendHealth(rssi = wifiInfo.rssi, wakeFalseAccepts = wakeWord.falseAcceptsSinceLastReport())
```

Every few minutes. Two fields carry most of the value: `rssi` identifies rooms with poor coverage — useful to hand back to the hotel as a report — and a rising `wakeFalseAccepts` almost always means the device sits next to a television.

PART VI

Shipping

19. Testing against a local service

```
# terminal 1
npm run gateway

# terminal 2 — sanity check without the app
npm run turn -- "turn on the lights"
npm run turn -- --device d_511a --trace "it's freezing and fix the shower"
```

on `d_412a` vs `d_511a`

Say this	Expect
`turn on the lights`	Fast, ~30 ms. `ack` then `speak`
`it's freezing in here`	Reads current state, sets 22.5 °C
`it's too cold and fix the shower`	Two actions, one reply
`set the temperature to 35 degrees`	Refused — test that you don't say "Done"
`I'd like a club sandwich`	Confirmation state + `screen` with the total
`yes`	Order placed
`I'm having chest pain`	Escalation wording + emergency number on screen
`close the curtains`	Different outcomes, same words

Failure cases to test deliberately

- Kill the gateway mid-conversation → offline badge appears, reconnect is jittered.
- Restart it → client recovers without an app restart.
- Use an unknown `deviceId` → `UNKNOWN_DEVICE`, and you stop retrying.
- Send 31 turns in an hour → `RATE_LIMITED`.
- Aeroplane mode during `THINKING` → no stuck spinner, no false confirmation.

20. Migration plan

Stage	Do	Result
1	Add `WoohooClient`. Wire `onSpeak` to existing TTS. Send transcripts from the existing STT path.	Working end to end alongside V1. Nothing removed yet.
2	Add `onAck` + phrase pack. Add the state machine and screen rendering.	Feels fast. This is the step guests notice.
3	Delete the Dialogflow integration and its intent handling.	Large net code deletion.
4	Add barge-in, device-side endpointing, confirmation and escalation UI.	Feature-complete client.
5	Certificate provisioning; **remove all cloud credentials**; retire socket.io.	Security posture fixed. Ship gate.
6	Offline mode + local grammar.	Survives WAN outages.
7	Move STT server-side; stream Opus.	Better accuracy, per-property vocabulary.

Stages 1–2 are a working demo. Stage 5 is the gate for any real deployment.

21. Review checklist

Connection

- [ ] Exactly one WebSocket, opened at app start, not per turn
- [ ] `hello` is always the first frame after open
- [ ] Reconnect uses exponential backoff with **full jitter**, capped at 60 s
- [ ] `UNKNOWN_DEVICE` stops retrying and shows a service state
- [ ] Keepalive interval set (20 s); read timeout is 0, not a finite value

Audio

- [ ] 500 ms pre-roll buffer flushed on wake, discarded otherwise
- [ ] Ring buffer never touches disk, in any build
- [ ] `ack` plays from cached audio, never synthesised
- [ ] Barge-in ducks, confirms, then stops — and stops locally first
- [ ] Real `asrConfidence` is sent, never a constant
- [ ] Endpoint threshold is remotely configurable per property

Behaviour

- [ ] The device never says "Done" without a `speak` from the server
- [ ] Confirmation state shows the total and never auto-confirms
- [ ] Escalation wording is rendered verbatim and does not auto-dismiss

- [ ] Every spoken reply is also on screen
- [ ] Offline mode announces once and states what still works

Security — ship gate

- [ ] **No cloud credentials anywhere in the APK**
- [ ] Device key is generated in hardware and is non-exportable
- [ ] `wss://` with pinning *and* a documented rotation path
- [ ] Hardware mic mute cuts the mic in the driver, not in software
- [ ] No transcript or audio is written to logs in a release build

---